Service shutdown report

Target: **siem (local)**
Recording started (t = 0): 2026-10-17 08:40:07 CEST (UTC+0200)
Report generated: 2026-10-17 08:40:13 CEST
Operation: **stop** · Recorded window: 6s · compose exit code: **0**

No warnings: no container exited with an error or had to be killed, and no service logged a line with an error word during the shutdown.

A tile's red badge counts every log line with an error word ("error", "fail…", "fault…", whatever its level) since the dashboard's log tail last started, including the 50 lines of scrollback it replays, and it keeps counting after the report ends; the Warnings here count only the lines logged during the recording. So the two numbers can differ, and a line logged after a screenshot shows up in a later one.

Resource usage recorded while the stack was taken down. Each dashed red vertical line marks an event (a container stopped or removed (*down* stops each container, then removes it)), numbered in a band above the graph. Container states come from *docker compose ps*, checked every ~1.5s, so event times are good to about a second. Events up to 1.5s after the first event of a number share it (in practice, the changes seen in one status check); the event key under the host graph lists what each number covers, with its time to a tenth of a second. The largest host CPU peak is called out with where it falls among the markers. **Figure 1** lists the containers of this stack, grouped as in the per-container graphs of section 2.

Host CPU and RAM were read from /proc every 0.25s, per-container CPU and memory were read from the containers' cgroup files every 0.25 s. A line is broken wherever no sample was taken for longer than a few sampling intervals, rather than drawn straight across, and a × marks where a container's line ends before the window does (it stopped, was removed, or a one-shot job exited). A container's line starts at its first sample, once it is really running. A container's stop time is when its cgroup disappeared (its processes exited), right after its line ends; *ps* only notices once Docker has also removed it, up to a couple of seconds later.

Reading the metrics. **SD** (standard deviation) is the typical distance of a value from the mean. The **95th percentile** is the level usage stayed at or below 95% of the time; a **peak** far above it is a short burst rather than sustained load. **P(≥x)** is the share of the recording spent at or above *x*: the chance that a randomly picked moment was that busy. "How unusual the peak is" counts SDs above the mean; more than about 3 is a rare spike.

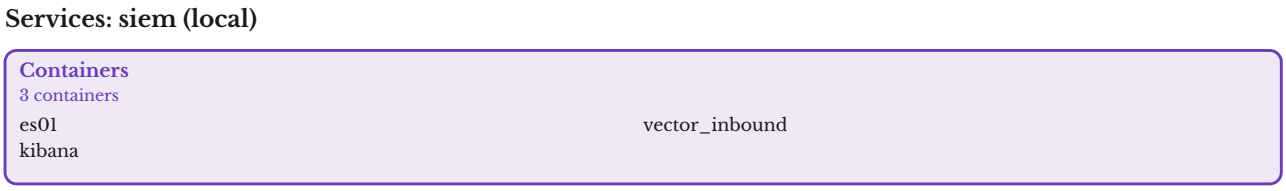
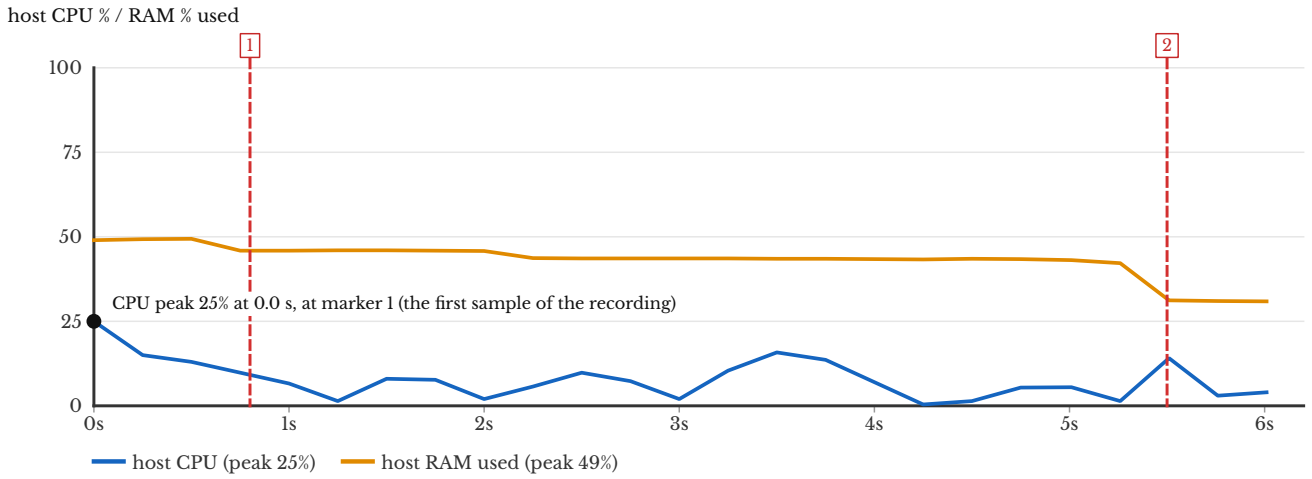


Figure 1. Containers of siem (local) grouped as in the per-container charts in section 2: each box lists exactly the containers those charts plot. The ids stack (reverse proxy, eshops, databases, Suricata) is a separate target with its own report; it ships its logs here via vector_inbound.

1. Overall system (host)



Event key (marker numbers on all graphs)

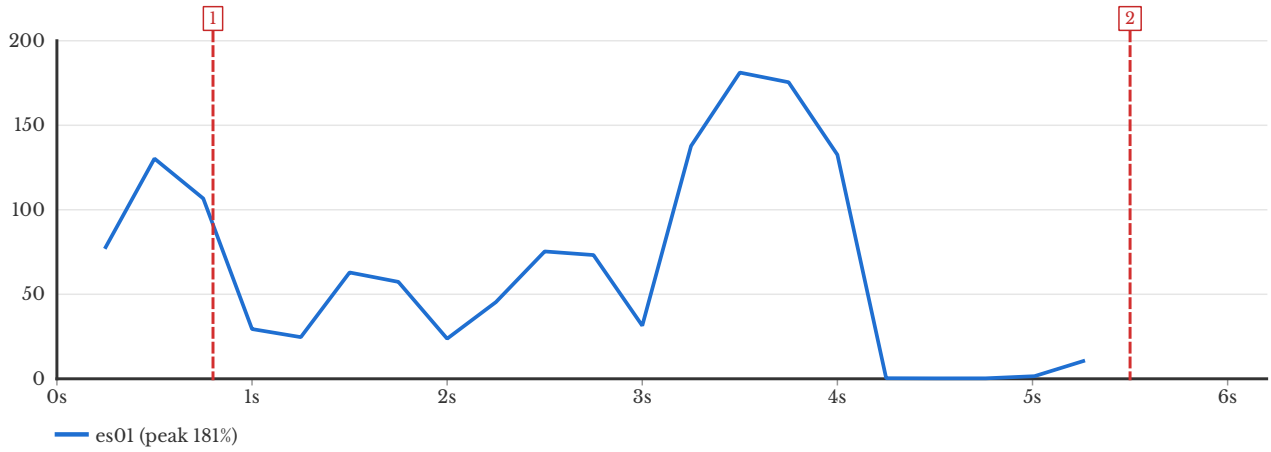
Marker	Elapsed	What happened
1	0.8 s–2.2 s	stopped and removed: kibana, vector_inbound
2	5.5 s	stopped and removed: es01

Probability metrics: host		
Host CPU, mean ± SD	**7.8% ± 5.9**	average over 25 samples; the SD is the typical swing around it
Host CPU, 95th percentile / peak	**15.6% / 25.0%**	95% of the time usage was at or below the first figure
Host CPU, how unusual the peak is	**2.9 SD above the mean**	more than ~3 SD is a rare spike rather than normal fluctuation
Host CPU, P(≥50%)	**0.0%**	chance a random moment of this recording was at least this busy
Host CPU, P(≥80%)	**0.0%**	chance a random moment of this recording was at least this busy
Host RAM used, mean ± SD	**43.2% ± 5.0**	average over 25 samples; the SD is the typical swing around it
Host RAM used, 95th percentile / peak	**49.2% / 49.4%**	95% of the time usage was at or below the first figure
Host RAM used, how unusual the peak is	**1.2 SD above the mean**	more than ~3 SD is a rare spike rather than normal fluctuation
Host RAM used, P(≥50%)	**0.0%**	chance a random moment of this recording was at least this busy
Host RAM used, P(≥80%)	**0.0%**	chance a random moment of this recording was at least this busy

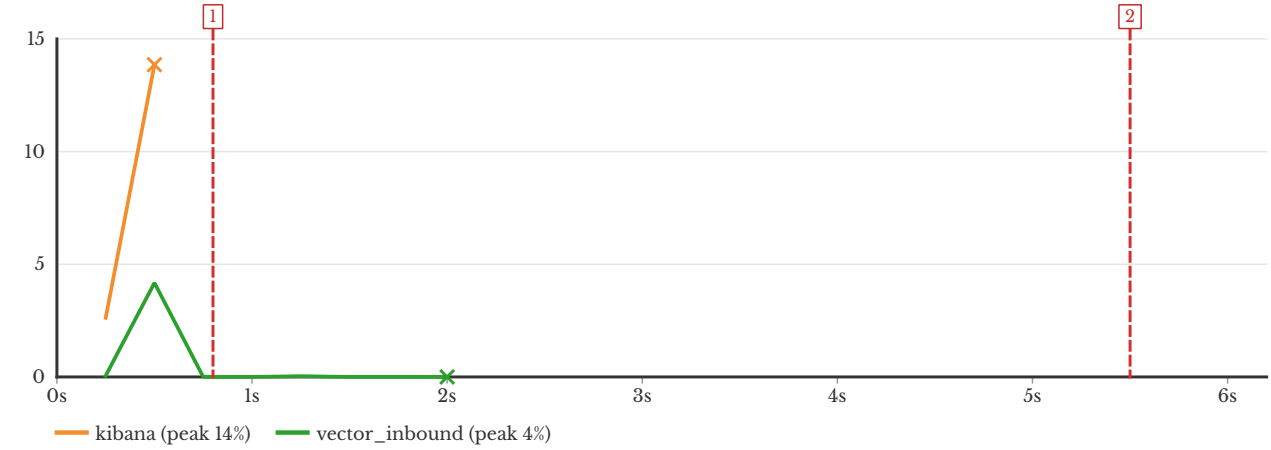
2. Per-container

Container CPU is in % of one core: 100% is one core fully busy, so a multi-threaded container goes above 100%. This host has 20 cores, so the ceiling is 2000%. When one or two containers dwarf the rest, they get a chart of their own and the others are drawn on their own scale.

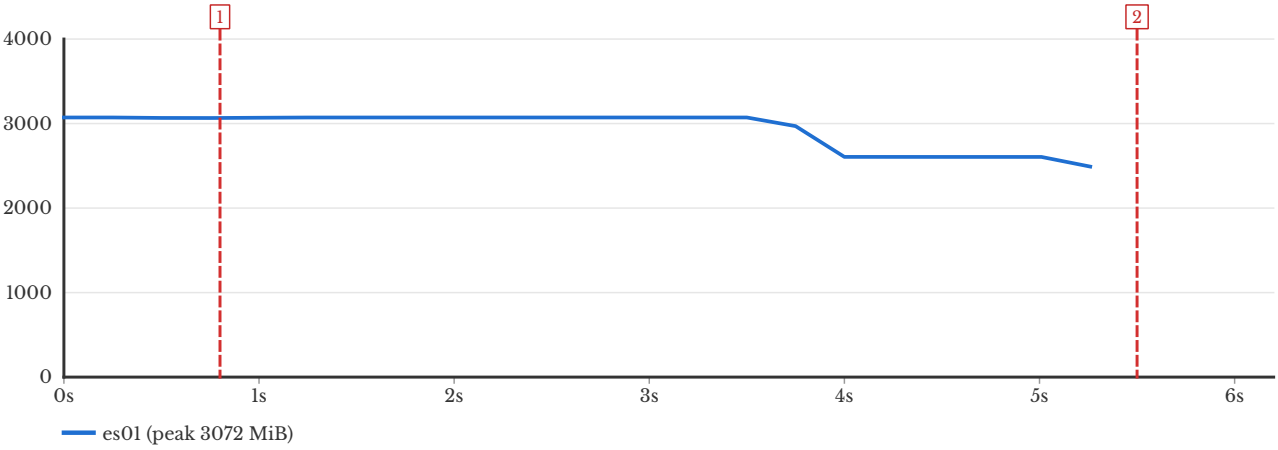
container CPU %, es01

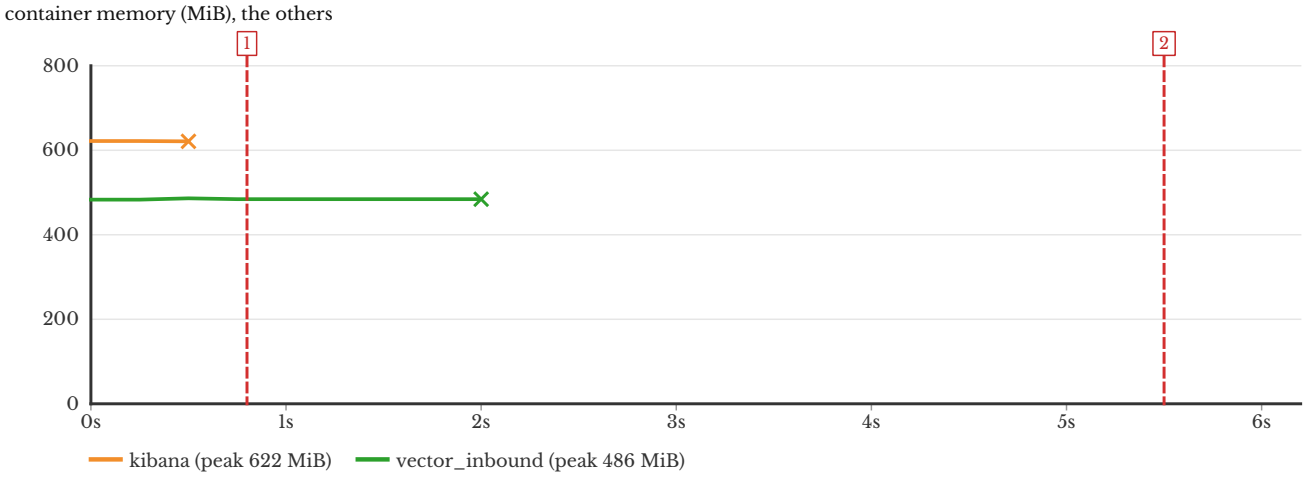


container CPU %, the others



container memory (MiB), es01





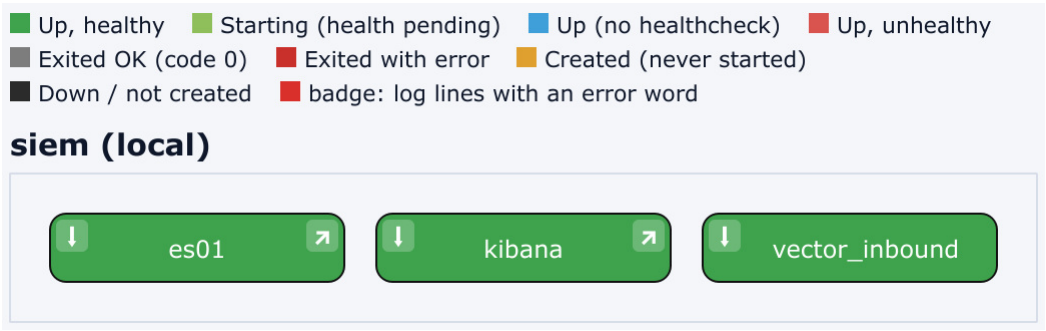
Probability metrics: per container						
Container	**Samples / over**	**CPU mean ± SD**	**CPU p95 / peak**	**P(CPU ≥ 50%)**	**Mem MiB mean ± SD**	**Mem MiB peak**
es01	22 / 5.3 s	65.6% ± 58.1	175.5% / 181.2%	52.4%	2934 ± 220	3072
kibana*	3 / 0.5 s	8.2% ± 7.9	13.3% / 13.9%	0.0%	622 ± 0	622
vector_inbound*	9 / 2.0 s	0.5% ± 1.5	2.7% / 4.2%	0.0%	484 ± 1	486

Each row covers only the samples taken while that container was running (count and the time between its first and last one). P(CPU ≥ 50%) is the share of those samples at half a core or more, i.e. the chance of catching it that busy at a random moment of its run. A peak far above the p95 means a short burst (typically start-up work or a first healthcheck) rather than sustained load. * fewer than 10 samples (the container ran only briefly while recorded): its figures describe a moment, not the whole run. **es01** sat at or near its 3072 MiB memory limit for 68% of its samples. The cgroup memory figure includes the page cache, so a database or Elasticsearch filling its limit is normally cached files (released under pressure), not a leak.

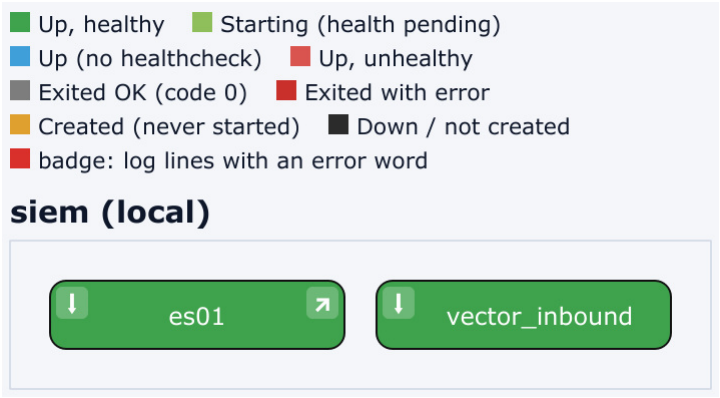
3. Global Visualization page as the stack changes state

Only this target's tiles, drawn from the same *docker compose ps* result the report logged at that moment, so tile colours and the event log agree. Badges are explained under Warnings.

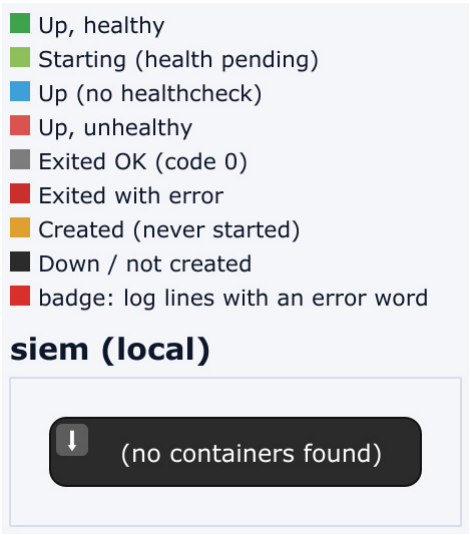
Before: stack idle / not yet changed (before t = 0)



Stopping: containers going down (at 1.6 s)



Stopped: down command finished (at 6.2 s)



Appendix: event log

The events behind the event key, one per row.

Marker	Elapsed	Event
1	0.8 s	kibana stopped and removed
1	2.2 s	vector_inbound stopped and removed
2	5.5 s	es01 stopped and removed